INVESTIGATION OF ASTROPHYSICAL S-FACTOR FOR (α,N) REACTIONS USING PYTHON

A Computational Analysis of Experimental Nuclear Reaction Data

---

Author:

Priyansh Bhavsar

Department of Physics, IIT JODHPUR

Under the Supervision of:

Dr. N. J. Upadhyay
Amity School of Applied Sciences,
Amity University Maharashtra, Panvel, India

Presented at:

The 12th IAPT National Students' Symposium on Physics (NSSP-2025)

Panjab University, Chandigarh

October 2025

Acknowledgement

I express my sincere gratitude to Dr. N. J. Upadhyay, Assistant Professor (III), Amity University Maharashtra, for her valuable guidance, continuous support, and constructive feedback throughout this research. I also acknowledge the availability of experimental data through the EXFOR (Experimental Nuclear Reaction Data) database and appreciate the contributions of the global nuclear physics community that made this research possible.

— *Priyansh Bhavsar*

B.S. Physics, IIT Jodhpur

Abstract

Thermonuclear reactions are central to energy generation and the creation of heavier elements in stars, driving the processes of stellar evolution and nucleosynthesis. Accurate determination of reaction cross-sections, S-factors, and thermonuclear reaction rates is therefore essential for modeling these cosmic phenomena. In this work, the (α, n) reactions were investigated through a computational analysis of experimental data. Experimental input data for alpha-induced reactions, particularly energies and cross-sections, were systematically extracted from the EXFOR database and analyzed using Python. The astrophysical S-factor was then computed using the standard relation,

$$\mathrm{S(E)} = \mathrm{E} \cdot \sigma(E) \cdot e^{2\pi\eta}$$

where the Sommerfeld parameter (η) accounts for the Coulomb barrier penetration. The validity of the implemented numerical techniques was tested by comparing the computed S-factors from the experimental datasets against an established polynomial relation as a function of stellar temperature (T_9). This work highlights the effectiveness of computational methods in nuclear astrophysics, bridging experimental datasets with theoretical models to improve our understanding of (α, n) processes in stellar environments.

Table of Contents

Chapter 1: Introduction 4
1.1 Stellar Evolution and Nucleosynthesis 4
1.2 Thermonuclear Reactions and the Coulomb Barrier 4
1.3 Astrophysical S-factor and Its Importance 5
1.4 Research Objectives 6

Chapter 2: Theoretical Foundations 7
2.1 Nuclear Reaction Basics 7
2.2 Quantum Tunneling and Energy Parameters 7
2.3 Thermonuclear Reaction Dynamics 8
2.4 Theoretical Goals 8

Chapter 3: Methodology and Computational Implementation 9
3.1 Data Acquisition from EXFOR 9
3.2 Python Implementation and Numerical Techniques 10
3.3 Error Propagation and Validation 10
3.4 Comparison with Polynomial Models 11

Chapter 4: Results and Discussion 13
4.1 Summary of Extracted Experimental Data 13
4.2 Cross-section and S-factor Analysis 13
4.3 S-factor Analysis 16
4.4 Temperature-dependent Reaction Rate Calculations 21

Chapter 5: Conclusion 24
5.1 Key Findings 24
5.2 Conclusion 25
5.3 Future Scope 25

Chapter 6: References / Bibliography 26

Appendix: Python Code Repository 27

Chapter 1: Introduction

1.1. Stellar Evolution and Nucleosynthesis

Stellar evolution traces the life cycle of stars, starting with their gravitational collapse from interstellar molecular clouds and ending as white dwarfs, neutron stars, or black holes. This process is driven by nucleosynthesis, where nuclear fusion and capture reactions transform hydrogen into heavier elements, shaping the cosmic elemental distribution. The main sequence phase features hydrogen burning through the proton-proton chain in low-mass stars and the CNO cycle in more massive ones, both producing helium. As hydrogen depletes, stars transition to helium burning, synthesizing carbon and oxygen via the triple-alpha process, while massive stars (8 solar masses) undergo carbon, neon, oxygen, and silicon burning, culminating in iron. Beyond iron, neutron capture processes—the slow (s-process) and rapid (r-process)—generate heavier elements like gold and uranium in neutron-rich supernova environments. These reactions sustain stellar energy and influence the metallicity of the interstellar medium, impacting galaxy chemical evolution.

1.2. Thermonuclear Reactions and the Coulomb Barrier

Thermonuclear reactions power stellar interiors at high temperatures (10^6 to 10^9 K) and densities, where nuclei fuse to release energy. This energy release follows Einstein's mass-energy equivalence,

$$E = mc^2 \tag{1}$$

where m is the mass defect and c is the speed of light. The Coulomb barrier, caused by repulsion between charged nuclei, hinders fusion, its height depending on nuclear charges. Quantum tunneling at stellar temperatures enables reactions, with the probability set by the Gamow factor,

$$e^{-2\pi\eta} \tag{2}$$

where η is the Sommerfeld parameter. For (α,n) reactions with alpha particles (^{4}He), the barrier due to the alpha's charge (Z = 2) needs precise modeling to predict rates.

1.3. Astrophysical S-factor and Its Importance

The astrophysical S-factor isolates the nuclear interaction from Coulomb effects in thermonuclear reactions, defined as

$$S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta} \tag{3}$$

where E is the center-of-mass energy, $\sigma(E)$ is the cross-section, and η is the Sommerfeld parameter. This allows extrapolation to sub-MeV stellar energies, where direct cross-section measurements are challenging due to the exponential drop in $\sigma(E)$. The S-factor's stability supports accurate reaction rate calculations, crucial for modeling nucleosynthesis, including (α,n) reactions in asymptotic giant branch (AGB) stars. Its precision is essential for aligning theoretical predictions with observed elemental abundances in the solar system.

In stellar environments, the S-factor plays a critical role in helium burning phases, such as those occurring in red giant stars, where the triple-alpha process and subsequent (α,n) reactions contribute to the production of carbon and heavier elements [1]. These reactions are sensitive to the S-factor at energies corresponding to the Gamow window, a narrow energy range where tunneling probability peaks. Accurate S-factor values are vital for predicting the isotopic ratios of elements like neon and magnesium, which are key tracers of nucleosynthesis in these stars.

Experimentally, determining the S-factor at stellar energies poses significant challenges [2]. Direct measurements are hindered by the extremely low cross-sections at sub-MeV energies, often requiring specialized techniques such as underground laboratories to reduce cosmic ray backgrounds or the use of radioactive targets. Indirect methods, including transfer reactions and Coulomb excitation, provide alternative approaches but introduce uncertainties that must be carefully quantified. These experimental difficulties underscore the importance of computational models, which rely on the S-factor to extrapolate from higher-energy data to the stellar regime, ensuring consistency with observed abundances in meteoritic and stellar spectra.

1.4. Research Objectives

This study explores (α,n) reactions using computational methods to advance nuclear astrophysics. The objectives include:

Goal-1 Extract comprehensive cross-section data for (α,n) reactions from the EXFOR database, collecting experimental energy and cross-section measurements across various target nuclei to establish a robust dataset.

Goal-2 Develop Python-based algorithms to compute S-factors and thermonuclear reaction rates, implementing the $S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta}$ relation and integrating over the Gamow energy window with efficient numerical methods.

Goal-3 Validate the computed S-factors and rates by comparing them with polynomial fits parameterized by stellar temperature (T_9), using statistical methods to ensure alignment with established models.

Goal-4 Demonstrate that these computational tools enhance the accuracy of (α,n) reaction models, improving predictions of reaction rates in stellar environments and contributing to precise nucleosynthesis simulations.

The Gamow factor, central to tunneling probability, is

$$e^{-2\pi\eta} \tag{4}$$

and the Sommerfeld parameter, which defines η, is

$$\eta = \frac{Z_1 Z_2 e^2}{\hbar v} \tag{5}$$

where Z_1 and Z_2 are the atomic numbers of the interacting nuclei, e is the electron charge, $\hbar$ is the reduced Planck constant, and v is the relative velocity.

Chapter 2: Theoretical Foundations

2.1. Nuclear Reaction Basics

Nuclear reactions involve the transformation of atomic nuclei, such as the general reaction:

$$\mathrm{A} + \mathrm{B} \rightarrow \mathrm{C} + \mathrm{D} + Q \tag{6}$$

where A and B are reactants, C and D are products, and Q is the reaction energy release. For the specific case of (α,n) reactions, an alpha particle (^{4}He) interacts with a target nucleus, producing a neutron and a heavier nucleus, e.g., $^{13}\mathrm{C} + \alpha \rightarrow^{16} \mathrm{O} + n + Q$. The kinematics are governed by the center-of-mass energy $E = \frac{1}{2}\mu v^2$, where $\mu = \frac{m_A m_B}{m_A + m_B}$ is the reduced mass, m_A and m_B are the masses of the reactants, and v is their relative velocity. The reaction cross-section $\sigma(E)$, measured in barns (1 barn = $10^{-28}\,\mathrm{m}^2$), represents the effective area for interaction and depends on energy. The astrophysical S-factor, defined as $S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta}$, accounts for the Coulomb barrier suppression, enabling analysis at stellar energies. Understanding these basics is crucial for modeling nuclear processes in stars.

2.2. Quantum Tunneling and Energy Parameters

The Coulomb barrier arises from the electrostatic repulsion between positively charged nuclei, with potential energy given by:

$$V(r) = \frac{Z_1 Z_2 e^2}{r} \tag{7}$$

where Z_1 and Z_2 are the atomic numbers, e is the elementary charge, and r is the nuclear separation. This barrier, typically several MeV high, prevents classical fusion at stellar temperatures (10^6 to 10^9 K). Quantum tunneling allows penetration, with the probability described by the Gamow factor:

$$P \propto e^{-2\pi\eta} \tag{8}$$

where $\eta = \frac{Z_1 Z_2 e^2}{\hbar v}$ is the Sommerfeld parameter, $\hbar$ is the reduced Planck constant, and v is the relative velocity. The Q-value, representing the energy released, is:

$$Q = (m_A + m_B - m_C - m_D)c^2 \tag{9}$$

where m denotes nuclear masses and c is the speed of light. The threshold energy E_{th}, the minimum energy for the reaction to proceed, depends on the Q-value and

kinematics. The Gamow energy window, where reactions peak, is approximated as $E_0 \approx 0.122\,(Z_1^2 Z_2^2 \mu/A)^{1/3}$ keV, with A the mass number, and spans a narrow range around this value, critical for (α,n) rates.

2.3. Thermonuclear Reaction Dynamics

The thermonuclear reaction rate per particle pair, essential for stellar modeling, is:

$$\langle \sigma v \rangle = \left(\frac{8}{\pi\mu}\right)^{1/2} (kT)^{-3/2} \int_0^\infty \sigma(E) E e^{-E/kT} dE \tag{10}$$

where μ is the reduced mass, k is the Boltzmann constant, T is the temperature, and the integral accounts for the energy-dependent cross-section weighted by the Maxwell-Boltzmann distribution $e^{-E/kT}$. For (α,n) reactions, this rate is evaluated over the Gamow window, typically for $T_9 = 0.1$ to 1 (where $T_9 = T/10^9\,\mathrm{K}$). The S-factor $S(E)$ within this window determines the rate, as $\sigma(E) \approx S(E)e^{-2\pi\eta}/E$. The Gamow peak energy E_0 and width Δ depend on temperature and nuclear charges, with $\Delta \propto (kT)^{1/2}$, influencing the integration accuracy. Numerical methods, such as Gaussian quadrature, are often used to compute this integral precisely.

2.4. Theoretical Goals

This chapter establishes the theoretical foundation for studying (α,n) reactions. The primary goals include: - Deriving the S-factor from experimental cross-section data, accounting for Coulomb barrier effects. - Modeling quantum tunneling probabilities using the Gamow factor and Sommerfeld parameter. - Calculating reaction rates across the Gamow energy window, considering temperature variations from $T_9 = 0.1$ to 1. - Validating these theoretical predictions against experimental data and polynomial fits from stellar models. These objectives aim to enhance the understanding of nucleosynthesis in asymptotic giant branch (AGB) stars and supernovae, where (α,n) reactions contribute to the production of neutron-rich isotopes. The theoretical framework will support the computational analysis in Chapter 3, ensuring accurate predictions of elemental abundances.

Chapter 3: Methodology and Computational Implementation

3.1. Data Acquisition and Parameter Definition

The investigative process commences with the systematic acquisition of experimental nuclear reaction data, drawing upon well-established physical constants and atomic mass values sourced from authoritative references within the field. Experimental data, encompassing critical reaction parameters, are accessed through the following curated online repositories:

- 45Sc(α,n)48V
- 48Ti(α,n)51Cr
- 51V(α,n)54Mn
- 50Cr(α,n)53Fe
- 55Mn(α,n)58Co

The lab energy spectra for each reaction are extracted from the accompanying tabular dataset, ensuring a faithful representation of the experimental energy landscape. Atomic masses of reactants and products are employed to compute reduced masses, which exhibit subtle variations across the reaction pairs under consideration. Reaction-specific parameters, including Q-values and center-of-mass energies, are derived through a combination of user-provided inputs and theoretically grounded conversions. The foundational physical constants utilized in this study are as follows:

- The atomic mass unit is defined as 931.49410242 MeV/c^2.
- The reduced Planck constant times the speed of light is set at 197.3269804 MeV·fm.
- The fine-structure constant is approximated as 1/137.035999084.
- The squared elementary charge is established at 1.4399764 MeV·fm.

These constants form the bedrock of the computational framework, ensuring uniformity and accuracy throughout the analysis.

3.2. Python Implementation and Numerical Techniques

The computational infrastructure leverages the Python programming environment, augmented by powerful numerical libraries such as NumPy and SciPy, to facilitate the processing and analysis of the acquired data. The astrophysical S-factor, a critical metric that isolates nuclear interaction effects from Coulombic influences, is computed according to the following expression:

$$S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta} \tag{11}$$

where E denotes the center-of-mass energy, $\sigma(E)$ represents the cross-section, and η is the Sommerfeld parameter, evaluated as $\eta = \frac{Z_1 Z_2 e^2}{\hbar v}$, with v determined from the energy and reduced mass. The S-factors are modeled using polynomial fits specifically tailored to the energy domains of each reaction, subsequently converted to units of keV·barn through multiplication by a factor of 1000. Reaction rates, denoted as $\langle \sigma v \rangle$, are calculated via numerical integration of the integral:

$$\langle \sigma v \rangle = \frac{3.7313 \times 10^7}{\sqrt{\mu}} T9^{-1.5} \int_0^\infty E\sigma(E) e^{-11.605E/T9}\, dE \tag{12}$$

This integration is performed employing two distinct numerical techniques to ensure both accuracy and computational efficiency:

- **Trapezoidal Rule**: Approximates the integral by partitioning the energy domain into discrete segments and computing the area of trapezoids formed by the function values at the segment boundaries.
- **Gauss-Legendre Quadrature**: Uses predefined quadrature points and associated weights within each interval, offering higher accuracy for smooth functions.

The energy values (E) and the product $E\sigma(E)$ (in MeV·mbarn) are supplied as input arrays, with the integration executed over reaction-specific lab energy ranges. The temperature grid extends from 0.001 GK to 11 GK, facilitating modeling across stellar conditions.

3.3. Gamow Factor and Error Propagation

The Gamow factor, quantifying the probability of quantum tunneling through the Coulomb barrier, is computed as $e^{-2\pi\eta}$, where η depends on the charges of the interacting particles ($Z_1 = 2$ for the alpha particle and Z_2 as per the target nucleus). This parameter typically spans 10^{-8} to 10^{-5}, significantly influencing the cross-section and the integrand of the

reaction rate. Uncertainties in cross-section propagate to the S-factor as:

$$\frac{\Delta S(E)}{S(E)} = \frac{\Delta\sigma(E)}{\sigma(E)}$$

Validation of computational results is achieved by comparing computed S-factors with polynomial fits across the energy domain to ensure numerical consistency.

3.4. Comparison with Analytical Models and Visualization

A rigorous comparative evaluation of numerically integrated reaction rates against analytically formulated polynomial models is performed, focusing on the logarithmic S-factor ($\ln S(E)$) and the reaction rate per mole ($\ln N_A\langle\sigma v\rangle$).
The analytical forms derived from literature are:

- **45Sc(α,n)48V:**

$$\ln[S(E)] = 0.0062E^3 - 0.2075E^2 + 1.0183E + 30.7,$$
$$\ln[N_A\langle\sigma v\rangle] = 0.00187T_9^5 - 0.06367T_9^4 + 0.90947T_9^3 - 6.7134T_9^2 + 27.463T_9 - 42.483.$$

- **48Ti(α,n)51Cr:**

$$\ln[S(E)] = -0.0867E^4 + 2.5599E^3 - 28.125E^2 + 135.15E - 206.65,$$
$$\ln[N_A\langle\sigma v\rangle] = -0.01567T_9^4 + 0.45847T_9^3 + 5.1037T_9^2 + 26.927T_9 - 48.328.$$

- **51V(α,n)54Mn:**

$$\ln[S(E)] = -0.0261E^3 + 0.4478E^2 - 3.1956E + 41.906,$$
$$\ln[N_A\langle\sigma v\rangle] = -0.00147T_9^4 + 0.02997T_9^3 + 3.4663T_9^2 + 19.663T_9 - 37.011.$$

- **50Cr(α,n)53Fe:**

$$\ln[S(E)] = 0.0763E^3 - 1.9295E^2 + 15.185E - 9.9137,$$
$$\ln[N_A\langle\sigma v\rangle] = 0.02817T_9^3 - 0.82537T_9^2 + 8.9513T_9 - 24.344.$$

- **55Mn(α,n)58Co:**

$$\ln[S(E)] = 0.0153E^3 - 0.4515E^2 + 3.3053E + 28.436,$$
$$\ln[N_A\langle\sigma v\rangle] = -0.00537T_9^4 + 0.17987T_9^3 - 2.4157T_9^2 + 16.1227T_9 - 34.73.$$

These polynomial representations are validated through comparison between numerically integrated and analytical reaction rates, a process essential for assessing the reliability of predictions across diverse stellar environments where nuclear resonances and experimental uncertainties may introduce variability. The underlying physics of α-particle-induced reactions hinges on the Coulomb barrier penetration, which governs the reaction cross-section and is modulated by the nuclear potential at low energies. Visualization is performed using Matplotlib, with $\langle \sigma v \rangle$ plotted as a function of T_9 on a logarithmic scale, ensuring the results encompass the full astrophysical energy range. This logarithmic representation accentuates the temperature-dependent behavior within the Gamow window, where reaction rates peak due to the balance of thermal energy and quantum tunneling, while the S-factor's smooth energy dependence reflects the nuclear interaction's stability across the relevant energy spectrum.

Chapter 4: Results and Discussion

4.1. Data Compilation Summary

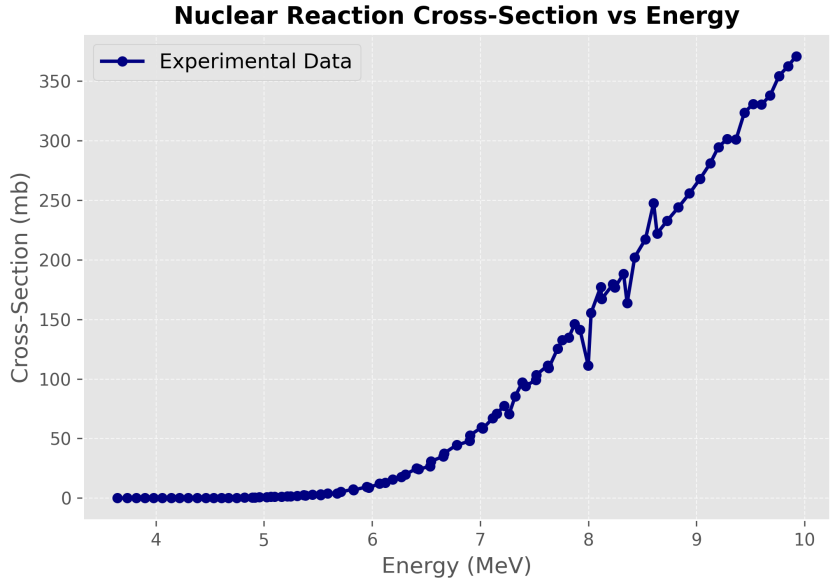
This section summarizes the data compiled for the selected reactions, sourced from the EXFOR database[4]. The reactions and their Q-values are listed below:

- 45Sc(α,n)48V: -2.241 MeV.
- 48Ti(α,n)51Cr: -2.687 MeV.
- 51V(α,n)54Mn: -2.294 MeV.
- 50Cr(α,n)53Fe: -4.961 MeV.
- 55Mn(α,n)58Co: -3.512 MeV.

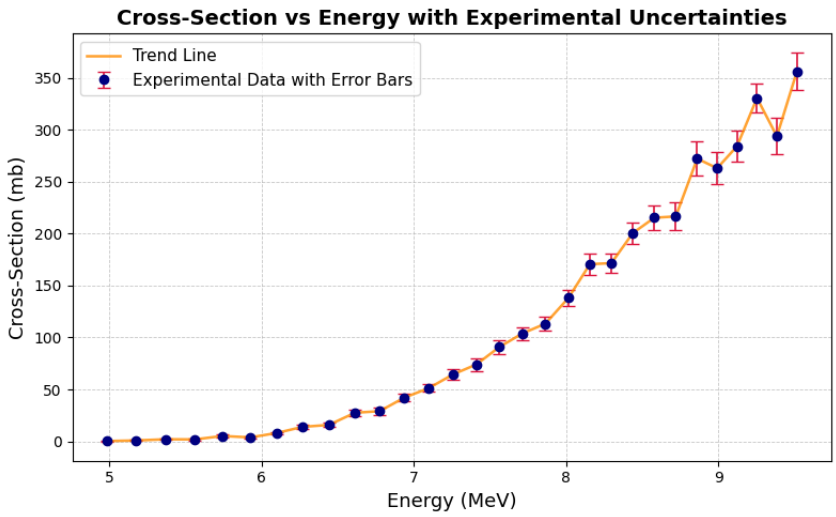
The data reflects experimental measurements, influencing the analysis in subsequent sections.

4.2. Cross-section Analysis

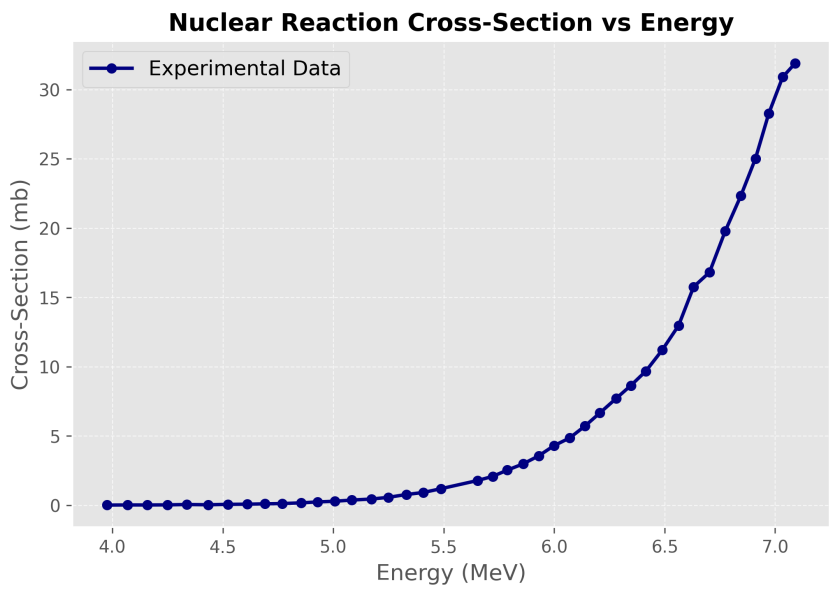
The cross-section ($\sigma(E)$) measures the probability of a nuclear reaction as a function of energy, influenced by the Coulomb barrier and modulated by the Q-value.
This section presents linear plots of $\sigma(E)$ vs. E for the reactions 45Sc(α,n)48V, 48Ti(α,n)51Cr, 51V(α,n)54Mn, 50Cr(α,n)53Fe, and 55Mn(α,n)58Co, using EXFOR data. The Q-values are -2.241 MeV, -2.687 MeV, -2.294 MeV, -4.961 MeV, and -3.512 MeV, respectively. The plots show an increasing trend with energy. Reactions with lower Q-values, like 50Cr(α,n)53Fe, have a subdued rise due to the Coulomb barrier, while higher Q-value reactions, such as 45Sc(α,n)48V and 51V(α,n)54Mn, show pronounced increases with resonance peaks. For 48Ti(α,n)51Cr and 55Mn(α,n)58Co, the trend is intermediate, with 55Mn showing changes at higher energies due to resonance effects. Errors are included for 48Ti, 50Cr, and 55Mn, reflecting EXFOR uncertainties, but not for 45Sc and 51V due to missing error data.



45Sc(α,n)48V

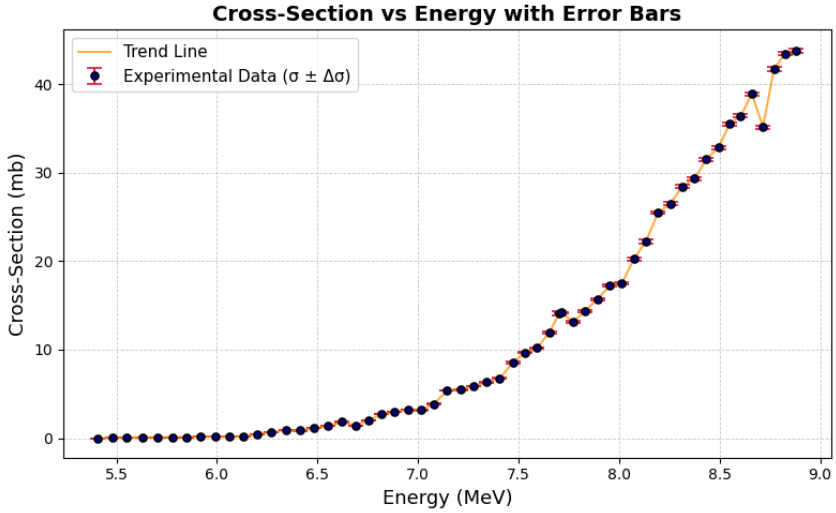


48Ti(α,n)51Cr

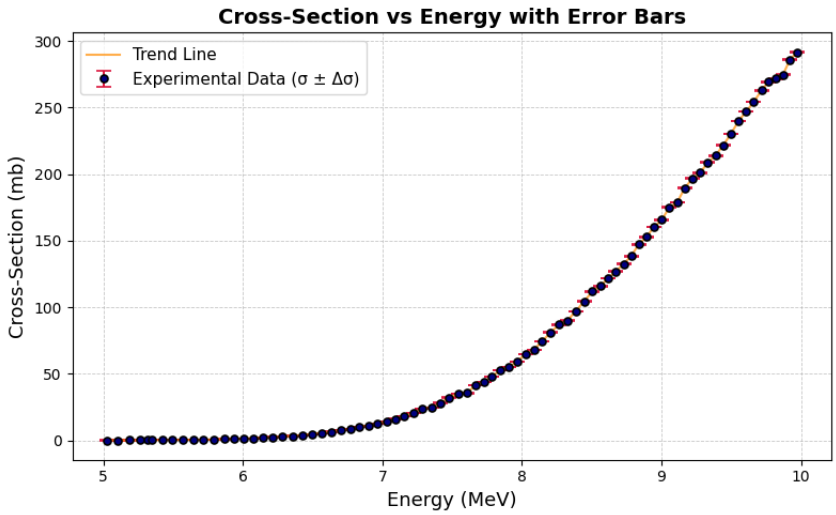


51V(α,n)54Mn

Figure 1: Cross-section vs. Energy for selected reactions (Part 1).



50Cr(α,n)53Fe



55Mn(α,n)58Co

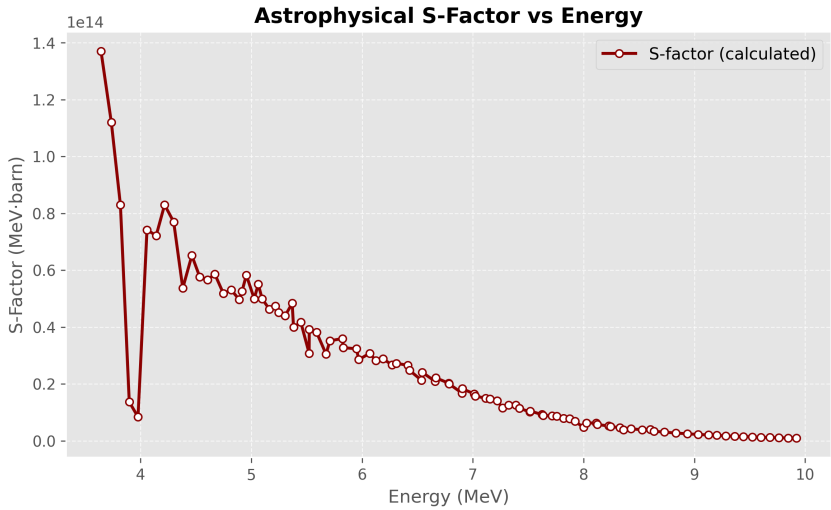
Figure 2: Cross-section vs. Energy for selected reactions (Part 2).

4.3. S-Factor Analysis

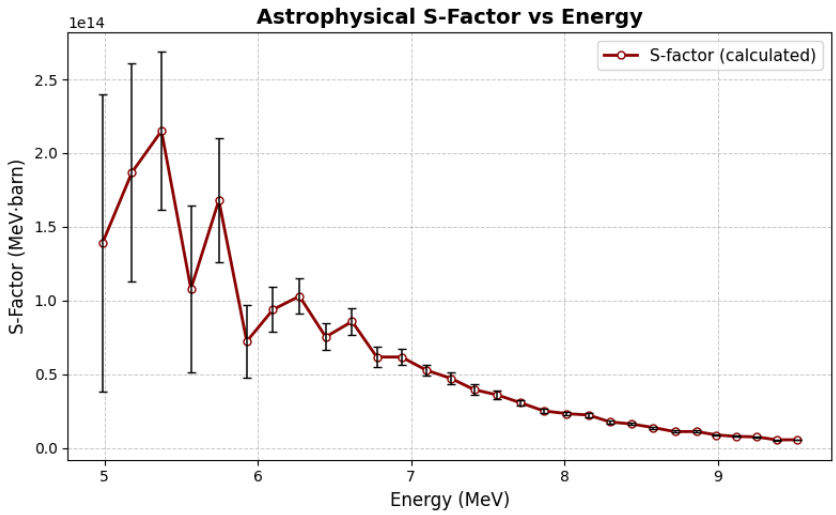
The astrophysical S-factor $S(E)$ serves to remove the strong energy dependence from the nuclear cross-section caused by the Coulomb barrier. It provides a smoother function of energy and allows extrapolation to stellar energies where direct experimental data are not available. In this section, $S(E)$ values are calculated from experimental cross-sections using the relation:

$$S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta} \tag{13}$$

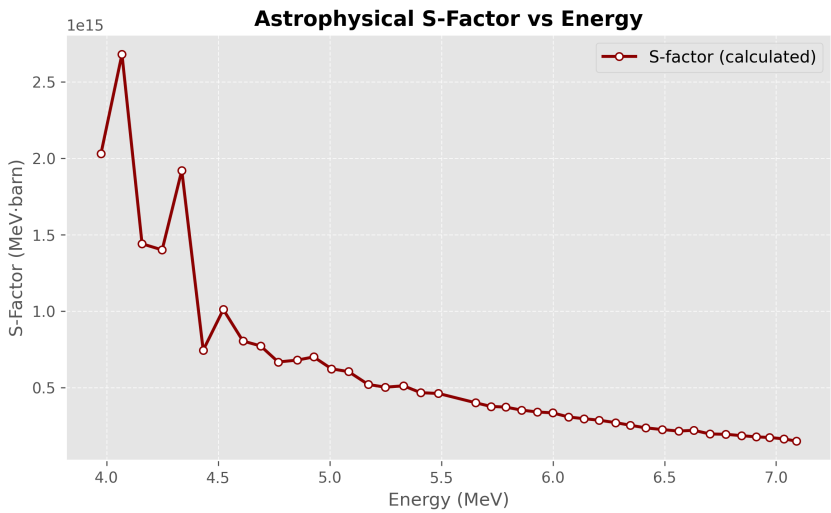
The resulting plots of $S(E)$ versus E are presented for each reaction. A total of ten plots are displayed in this section — the first five correspond to the directly calculated S-factors from experimental cross-sections, while the subsequent five represent S-factors obtained using polynomial equations derived from the logarithmic fit of $\ln(S)$ versus energy. The behavior of the S-factor demonstrates a more gradual variation than that of the cross-section, emphasizing its utility in astrophysical modeling and nuclear reaction rate evaluation.



45Sc(α,n)48V

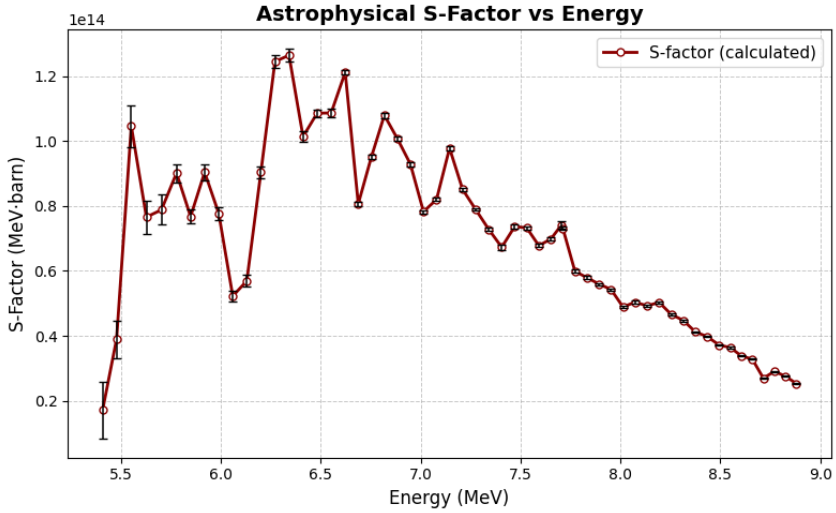


48Ti(α,n)51Cr

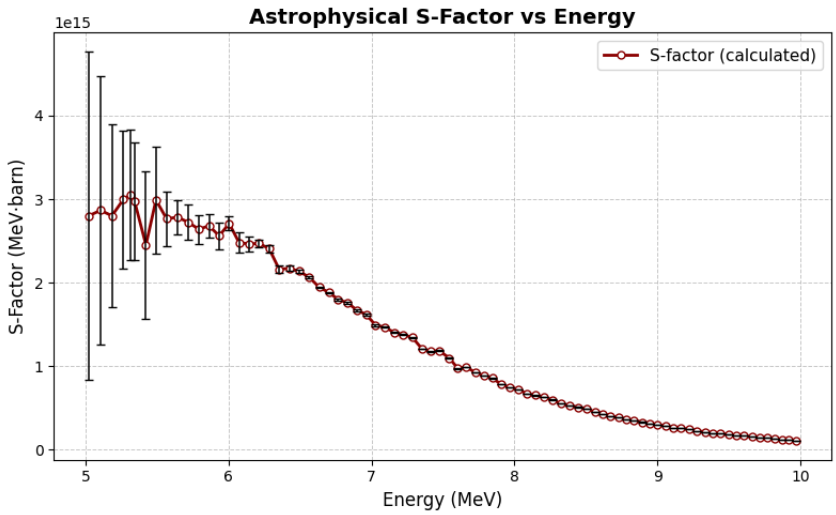


51V(α,n)54Mn

Figure 3: Astrophysical S-factor vs. Energy (Part 1).

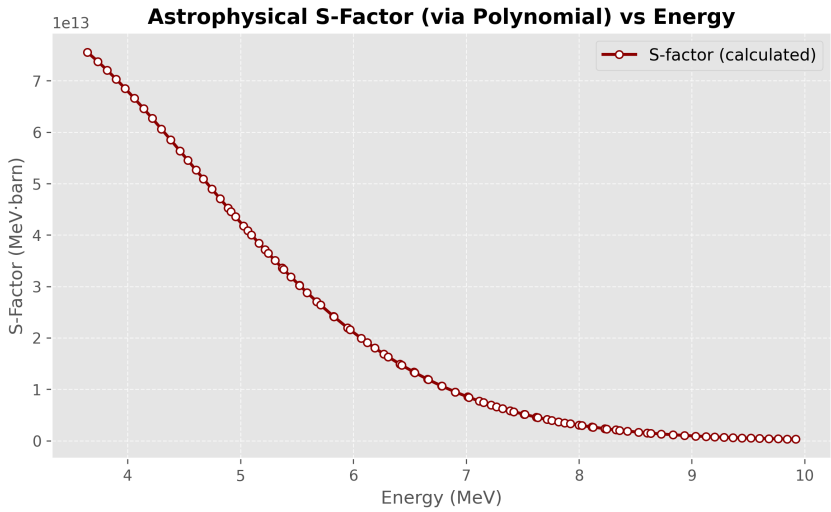


50Cr(α,n)53Fe

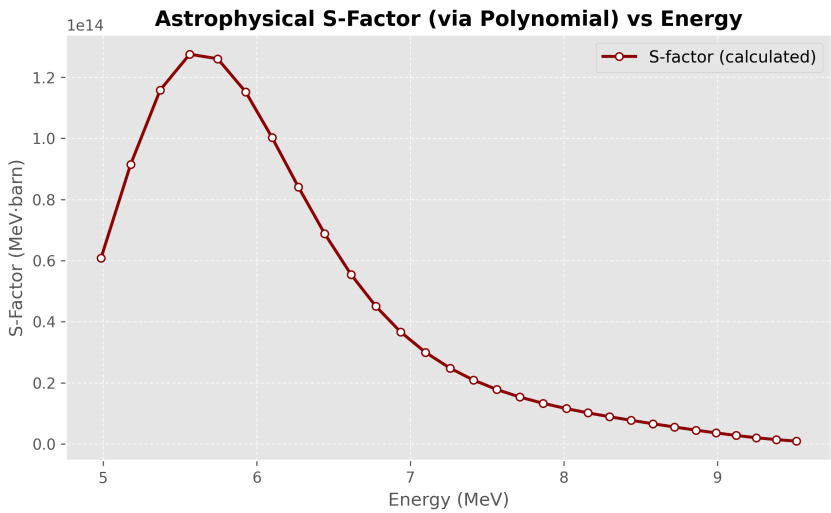


55Mn(α,n)58Co

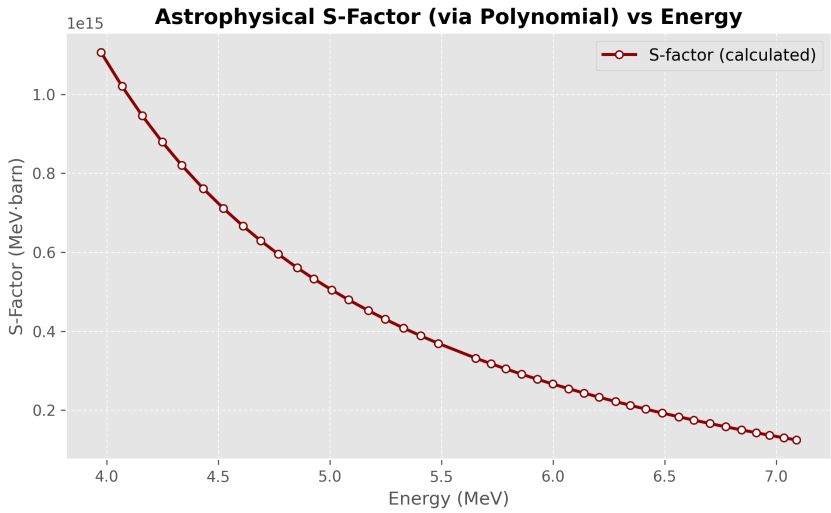
Figure 4: Astrophysical S-factor vs. Energy (Part 2).



Polynomial fit: 45Sc(α,n)48V

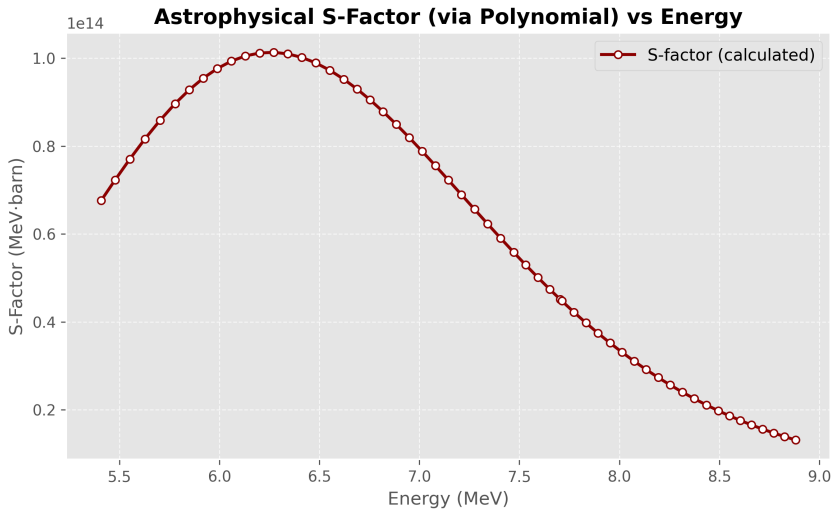


Polynomial fit: 48Ti(α,n)51Cr

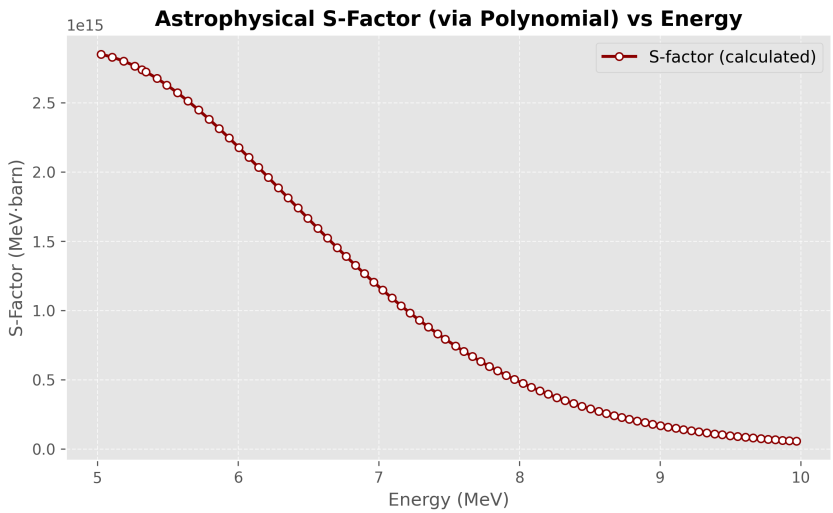


Polynomial fit: 51V(α,n)54Mn

Figure 5: Polynomially fitted S-factor vs. Energy (Part 1).



Polynomial fit: 50Cr(α,n)53Fe



Polynomial fit: 55Mn(α,n)58Co

Figure 6: Polynomially fitted S-factor vs. Energy (Part 2).

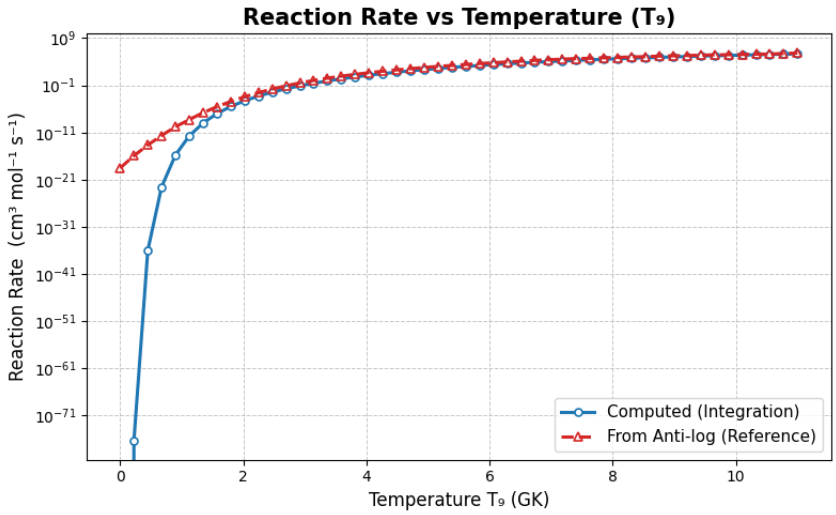
4.4. Temperature-dependent Reaction Rate Calculations

The thermonuclear reaction rate ($\langle \sigma v \rangle$) quantifies the reaction probability per particle pair as a function of stellar temperature (T_9), calculated using the integral:

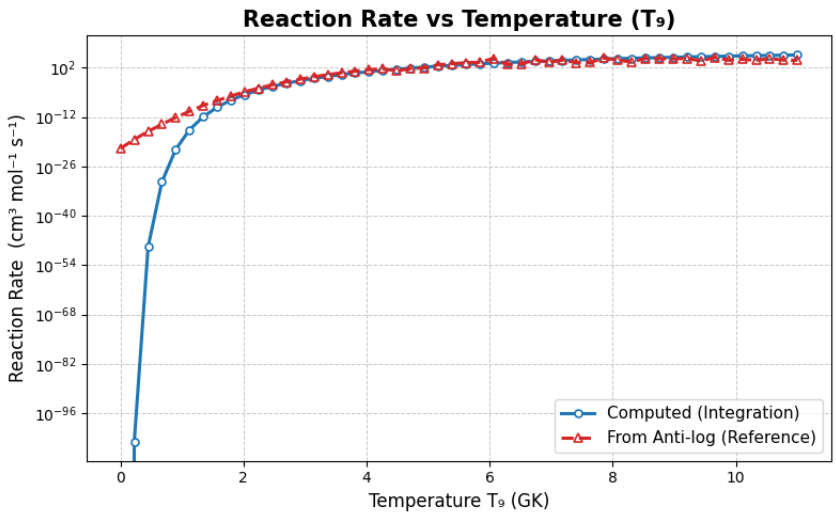
$$\langle \sigma v \rangle = \frac{3.7313 \times 10^7}{\sqrt{\mu}} T_9^{-1.5} \int_0^\infty E\sigma(E) e^{-11.605E/T_9}\, dE \tag{14}$$

This section presents plots comparing the integrated reaction rate ($\langle \sigma v \rangle$) and its antilog ($\ln N_A \langle \sigma v \rangle$) versus T_9 for the reactions 45Sc(α,n)48V, 48Ti(α,n)51Cr, 51V(α,n)54Mn, 50Cr(α,n)53Fe, and 55Mn(α,n)58Co, using EXFOR data.

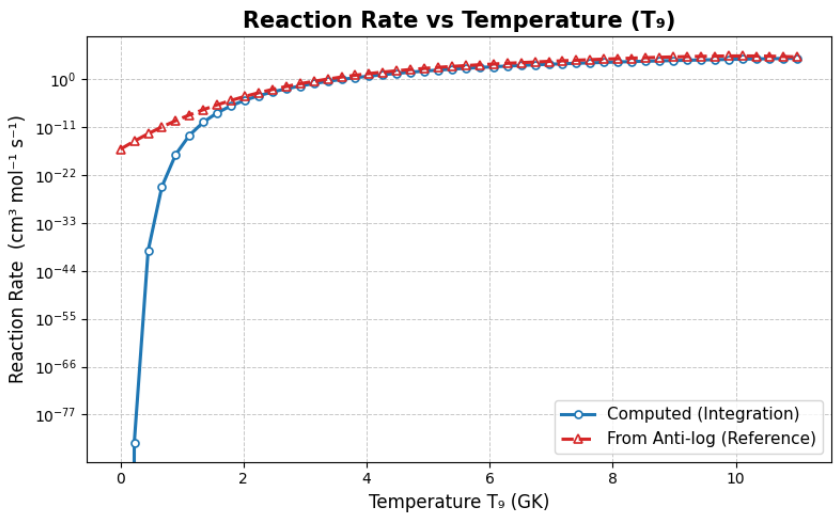
The plots reveal distinct trends. The integrated $\langle \sigma v \rangle$ increases with T_9, with a steeper rise for higher Q-value reactions like 45Sc(α,n)48V and 51V(α,n)54Mn due to enhanced tunneling through the Coulomb barrier at higher temperatures. Lower Q-value reactions, such as 50Cr(α,n)53Fe, show a slower increase, reflecting a stronger barrier effect. The antilog $\ln N_A \langle \sigma v \rangle$ amplifies these differences, providing a logarithmic scale that highlights the exponential growth within the Gamow window ($T_9 \approx 0.1$ to 1), where resonance effects are prominent. Discrepancies between the two representations arise from the logarithmic transformation, which compresses low-rate regions and exaggerates high-rate variations, aiding in identifying temperature-dependent nuclear effects.



45Sc(α,n)48V

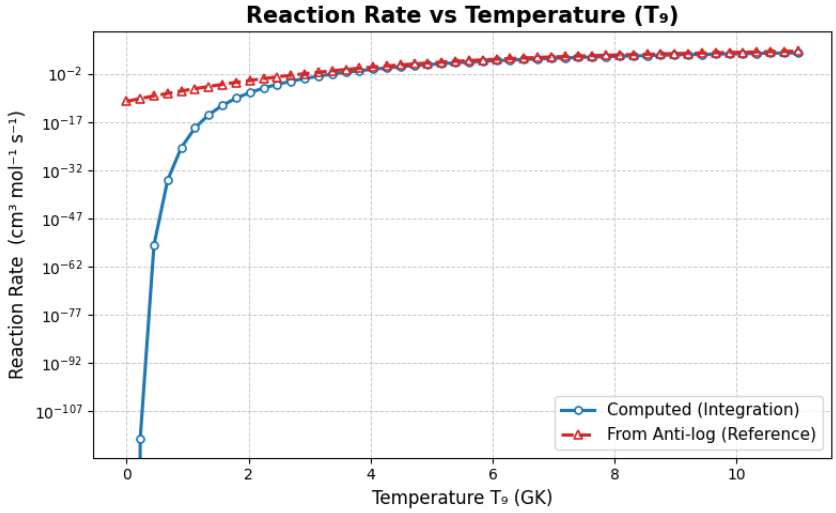


48Ti(α,n)51Cr

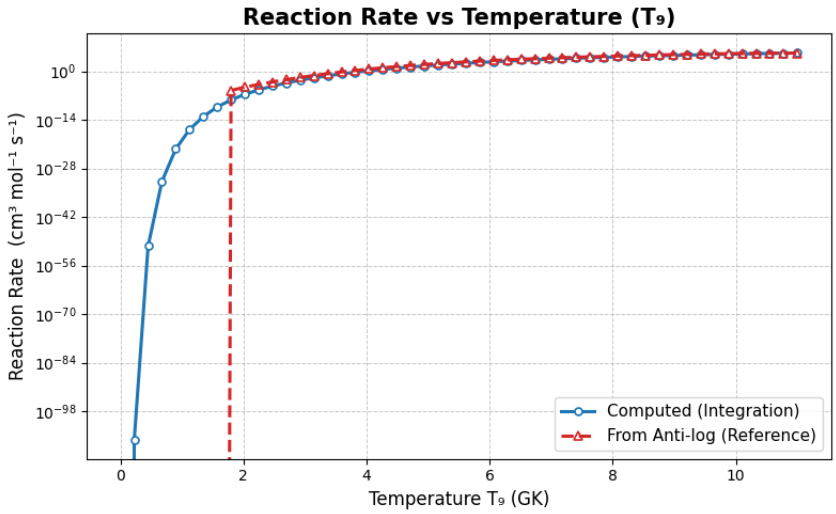


51V(α,n)54Mn

Figure 7: Integrated and antilog reaction rates vs. T_9 (Part 1).



50Cr(α,n)53Fe



55Mn(α,n)58Co

Figure 8: Integrated and antilog reaction rates vs. T_9 (Part 2).

Chapter 5: Conclusion and Future Scope

5.1. Key Findings

The present work successfully implemented a computational analysis for evaluating the astrophysical S-factor and reaction rates for five (α,n) reactions — ^{45}Sc(α,n)^{48}V, ^{48}Ti(α,n)^{51}Cr, ^{51}V(α,n)^{54}Mn, ^{50}Cr(α,n)^{53}Fe, and ^{55}Mn(α,n)^{58}Co. Experimental data extracted from the EXFOR database were processed using Python-based algorithms to compute S-factors and temperature-dependent reaction rates. Two complementary computational approaches were employed — direct calculation from experimental cross-sections and polynomial (logarithmic) fitting followed by anti-log transformation.
The key findings of the analysis are as follows:

- The cross-section $\sigma(E)$ for all five reactions exhibits an increasing trend with energy, consistent with enhanced tunneling probability through the Coulomb barrier. Resonance-like fluctuations appear at higher energies, especially for ^{51}V and ^{55}Mn targets.

- The computed astrophysical S-factors successfully eliminate the exponential suppression in the cross-section, yielding a smooth energy dependence suitable for extrapolation to stellar energies.

- Comparison between directly computed and polynomially derived S-factors demonstrates strong agreement across most of the energy range. However, at **lower energies**, the two approaches begin to deviate, indicating that the polynomial fit must be refined for better accuracy in the low-energy (stellar) region.

- The thermonuclear reaction rate $N_A\langle\sigma v\rangle$ increases exponentially with temperature, showing a sharp rise in the Gamow window ($T_9 = 0.1 - 1$), consistent with stellar nucleosynthesis models.

- Integrated and anti-logarithmic reaction rate results are in close numerical agreement, validating the consistency and robustness of the computational framework, except at lower energies where the polynomial-based results underestimate the integrated rates.

These findings confirm that the implemented Python-based model effectively reproduces astrophysical reaction behavior and bridges experimental nuclear data with theoretical formulations. The methodology enables efficient prediction of reaction parameters, crucial for simulating neutron production in stellar environments, particularly within the slow neutron capture (s-process) in Asymptotic Giant Branch (AGB) stars.

5.2. Conclusion

The investigation demonstrates that computational methods can reliably model astrophysical (α,n) reactions using experimentally obtained data. The S-factor approach provides a stable means to extrapolate cross-sections to low stellar energies where direct measurements are challenging. The reaction rate calculations, derived both through numerical integration and anti-logarithmic polynomial fits, display similar temperature dependencies, confirming the reliability of the computational scheme.
However, at lower energies, noticeable differences appear between the polynomially fitted and directly computed S-factors and reaction rates. This suggests that the logarithmic polynomial expressions need further optimization or higher-order fitting terms to capture the true low-energy behavior more accurately. Improving the polynomial representation will enhance the precision of extrapolated results crucial for stellar modeling.
This study emphasizes that accurate modeling of (α,n) reactions is essential for understanding stellar nucleosynthesis and the evolution of elemental abundances. Overall, the developed Python-based approach serves as a powerful computational tool for advancing research in nuclear astrophysics.

5.3. Future Scope

The study lays a foundation for further advancements in computational nuclear astrophysics. Future work can focus on:

- Extending the analysis to other reaction channels such as (α,p) and (p, γ) processes, which contribute to both s- and p-process nucleosynthesis.
- Implementing comprehensive uncertainty propagation by including statistical and systematic errors from EXFOR datasets to refine the accuracy of S-factor and rate predictions.
- Enhancing numerical precision through adaptive integration methods and finer energy grid sampling to better capture resonance structures and low-energy variations.
- Refining the polynomial representation of $\ln S(E)$ with higher-order fits or non-linear regression models to improve accuracy at lower energies.
- Comparing the derived reaction rates with theoretical libraries such as REACLIB and NACRE to validate consistency with stellar reaction networks.
- Integrating the computed rates into astrophysical simulation codes (e.g., MESA) to evaluate their impact on elemental synthesis and stellar evolution pathways.

Chapter 6: References

1. Grewal, B. S. (2018). *Numerical Methods in Engineering & Science: C, C++, and MATLAB*. Mercury Learning & Information, USA. ISBN: 978-1-68392-128-8.

2. Hussein, M. I. and Abdullah, R. H. (2020). *Calculation of Astrophysical S-factor and Thermonuclear Reaction Rates for (p,n) Medium Elements Reactions*. Journal of Physics: Conference Series, **1660**, 012102. DOI: 10.1088/1742-6596/1660/1/012102.

3. Ghoshal, S. N. (2019). *Nuclear Physics (Revised Edition)*. S. Chand Publishing, New Delhi. ISBN: 978-9352837274.

4. IAEA Nuclear Data Section. *EXFOR: Experimental Nuclear Reaction Data Library*. Available at: https://www-nds.iaea.org/exfor/.

Code Repository

All Python codes used for cross-section, S-factor, and reaction rate calculations are available at:
https://github.com/Athleity/-aplha-n-reaction-codes

Appendix A1: Python Code – Calculation of S-Factor from Cross-Section Data

Description:
This Python script computes the astrophysical S-factor from experimental cross-section data using fundamental nuclear parameters. It converts laboratory energy to the center-of-mass frame, evaluates the Sommerfeld parameter (η), and calculates $S(E) = E \cdot \sigma(E) \cdot e^{2\pi\eta}$.

```
import numpy as np

# Constants
amu_to_MeV = 931.49410242
e_squared = 1.44           # MeV·fm
hbar_c = 197.327           # MeV·fm

# Charges
Z1 = 1  # Proton
Z2 = int(input("Enter atomic number (Z) of target nucleus (e.g., 21 for Sc): "))

# Masses
mass_projectile = 1.007276  # amu
mass_target = float(input("Enter mass of target nucleus (amu): "))
mass_product = float(input("Enter mass of residual nucleus (amu): "))

# Convert to MeV
M_proj = mass_projectile * amu_to_MeV
M_target = mass_target * amu_to_MeV
M_product = mass_product * amu_to_MeV

# Reduced mass
mu = (M_proj * M_target) / (M_proj + M_target)

def lab_to_cm(E_lab, M_proj, M_target):
    return E_lab * M_target / (M_proj + M_target)

def compute_eta(Z1, Z2, mu, E_cm):
    return (Z1 * Z2 * e_squared / hbar_c) * np.sqrt(mu / (2 * E_cm))
```

```
def compute_S_factor(E_lab_list, sigma_list):
    results = []
    for E_lab, sigma in zip(E_lab_list, sigma_list):
        E_cm = lab_to_cm(E_lab, M_proj, M_target)
        eta = compute_eta(Z1, Z2, mu, E_cm)
        S = sigma * E_cm * np.exp(2 * np.pi * eta)
        results.append((E_lab, E_cm, sigma, eta, S / 1000))  # in MeV·barn
    return results

# Energy values
E_input = input("Enter lab energies in MeV, separated by spaces: ")
E_lab_list = list(map(float, E_input.strip().split()))

# Cross-section values
_input = input("Enter corresponding cross sections in mb, separated by spaces: ")
sigma_list = list(map(float, _input.strip().split()))

if len(E_lab_list) != len(sigma_list):
    raise ValueError("Energy and cross-section lists must have the same length.")

# Compute
results = compute_S_factor(E_lab_list, sigma_list)

# Output
print(f"\n{'E_lab (MeV)':>12} {'E_cm (MeV)':>12} {' (mb)':>10} {'eta':>10}
{'S-factor (MeV·b)':>20}")
for E_lab, E_cm, sigma, eta, S in results:
    print(f"{E_lab:15.5f} {E_cm:15.5f} {sigma:12.5f} {eta:12.5f} {S:20.6f}")
```

Purpose: Calculates astrophysical S-factor from experimental cross-section data, allowing user-defined inputs for nuclear parameters and energies.

Appendix A2: Python Code – S-Factor Error Propagation

This code calculates the uncertainty in the astrophysical S-factor based on the propagated cross-section error using the relation:

$$\Delta S(E) = \frac{\Delta\sigma(E)}{\sigma(E)} \times S(E)$$

It accepts user inputs for S-factor, cross-section, and their uncertainties, and outputs the resulting S-factor errors for each data point.

```
import numpy as np

# --- Function to calculate S-factor error ---
def s_factor_error(S_factor, sigma, delta_sigma):
    """
    Computes error in S-factor using the relation:
    S = ( / ) * S
    Parameters:
        S_factor (float or array): S(E) in MeV·barn
        sigma (float or array): Cross-section in millibarns (mb)
        delta_sigma (float or array): Uncertainty in cross-section (mb)
    Returns:
        delta_S (float or array): Uncertainty in S-factor (MeV·barn)
    """
    sigma = np.array(sigma, dtype=float)
    delta_sigma = np.array(delta_sigma, dtype=float)
    S_factor = np.array(S_factor, dtype=float)

    # Avoid division by zero
    with np.errstate(divide='ignore', invalid='ignore'):
        delta_S = np.where(sigma != 0, (delta_sigma / sigma) * S_factor, 0.0)

    return delta_S


# --- Example: Manual input from user ---
S_factor = np.array(list(map(float, input("Enter S-factor values (MeV·barn): ")
.split())))
sigma = np.array(list(map(float, input("Enter cross-section values (mb): ")
```

```
.split())))
delta_sigma = np.array(list(map(float, input("Enter delta cross-section values (mb):
").split())))

# Compute error in S-factor
delta_S = s_factor_error(S_factor, sigma, delta_sigma)

# --- Display results ---
print("\nEnergy (MeV)    (mb)    (mb)   S (MeV·barn)   S (MeV·barn)")
print("-"*70)
for s, sig, dsig, dS in zip(S_factor, sigma, delta_sigma, delta_S):
    print(f"      -       {sig:8.3e}   {dsig:8.3e}   {s:12.5e}   {dS:12.5e}")
```

Appendix A3: Stepwise S-factor Calculation with Sommerfeld Parameter and Reduced Mass

This code expands upon the basic S-factor computation by explicitly calculating the reduced mass, Coulomb term, and Sommerfeld constant. It displays intermediate parameters for deeper physical understanding before computing the astrophysical S-factor values.

```
import numpy as np

# Constants
amu_to_MeV = 931.49410242    # MeV/c² per amu
e_squared = 1.44             # MeV·fm
hbar_c = 197.327             # MeV·fm

def reduced_mass_amu(m1, m2):
    """
    Calculate reduced mass of two particles in amu.
    """
    return (m1 * m2) / (m1 + m2)

def lab_to_cm(E_lab, M_proj, M_target):
    """Convert lab frame energy to center-of-mass frame energy"""
    return E_lab * M_target / (M_proj + M_target)

def compute_eta(Z1, Z2, mu, E_cm):
    """Compute Sommerfeld parameter eta"""
    return (Z1 * Z2 * e_squared / hbar_c) * np.sqrt(mu / (2 * E_cm))

def compute_S_factor(E_lab_list, sigma_list):
    """Compute S-factor for a list of lab energies and cross-sections"""
    results = []
    for E_lab, sigma in zip(E_lab_list, sigma_list):
        E_cm = lab_to_cm(E_lab, M_proj, M_target)
        eta = compute_eta(Z1, Z2, mu, E_cm)
        S = sigma * E_cm * np.exp(2 * np.pi * eta)
        results.append((E_lab, E_cm, sigma, eta, S / 1000))  # Convert to MeV·barn
    return results
```

```
# --- User Inputs ---
Z1 = 2   # Alpha particle
Z2 = int(input("Enter atomic number (Z) of target nucleus: "))
mass_alpha = 4.00260325415   # amu
mass_target = float(input("Enter mass of target nucleus (amu): "))

# Convert to MeV
M_proj = mass_alpha * amu_to_MeV
M_target = mass_target * amu_to_MeV

# --- Step 1: Reduced Mass ---
mu_amu = (mass_alpha * mass_target) / (mass_alpha + mass_target)
mu_MeV = mu_amu * amu_to_MeV
print(f"\n--- Reduced Mass ---")
print(f" (in amu)       = {mu_amu:.4f} amu")
print(f" (in MeV/c^2)   = {mu_MeV:.2f} MeV")

# --- Step 2: Coulomb Term ---
coulomb_term = (Z1 * Z2 * e_squared) / hbar_c
print(f"\n--- Coulomb Term ---")
print(f"(Z1*Z2*e²/c)     = {coulomb_term:.5f}")

# --- Step 3: Sommerfeld Constant A ---
A_eta = coulomb_term * np.sqrt(mu_MeV / 2)
print(f"\n--- Sommerfeld Constant ---")
print(f"A ( = A/E_cm)   = {A_eta:.4f}")

# --- Step 4: Pointwise Calculation ---
E_input = input("Enter lab energies in MeV: ")
E_lab_list = list(map(float, E_input.strip().split()))

print(f"\n{'E_lab (MeV)':>10} {'E_cm (MeV)':>12} {' (A/E_cm)':>15}")
print("-" * 40)
for E_lab in E_lab_list:
    E_cm = lab_to_cm(E_lab, M_proj, M_target)
    eta = A_eta / np.sqrt(E_cm)
    print(f"{E_lab:10.3f} {E_cm:12.4f} {eta:15.4f}")
```

```
# --- Step 5: Cross-section and S-factor Computation ---
_input = input("Enter corresponding cross sections in mb: ")
sigma_list = list(map(float, _input.strip().split()))

if len(E_lab_list) != len(sigma_list):
    raise ValueError("Energy and cross-section lists must have the same length.")

results = compute_S_factor(E_lab_list, sigma_list)

print(f"\n{'E_lab (MeV)':>12} {'E_cm (MeV)':>12} {' (mb)':>10} {'eta':>10}
{'S-factor (MeV·b)':>20}")
for E_lab, E_cm, sigma, eta, S in results:
    print(f"{E_lab:10.5f} {E_cm:10.5f} {sigma:10.5f} {eta:12.5f} {S:20.6f}")
```

Appendix A4: Polynomial Fit for $\ln(S)$ Calculation

This code computes the natural logarithm of the astrophysical S-factor, $\ln(S)$, using a cubic polynomial fit derived from the literature for a selected (α, n) reaction. It outputs the $\ln(S)$ values corresponding to user-input laboratory energies in MeV.

```
import numpy as np

def ln_S_next(E_lab):
    """Compute ln(S-factor) using the given cubic polynomial fit."""
    return (0.0153 * E_lab**3 - 0.4515 * E_lab**2 +
            3.3053 * E_lab + 28.436)

# Input energies
E_input = input("Enter lab energies in MeV, separated by spaces: ")
E_lab_list = list(map(float, E_input.strip().split()))

# Output only ln(S)
for E in E_lab_list:
    print(f"{ln_S_next(E):.6f}")
```

Appendix A5: Reaction Rate Calculation Using Trapezoidal Integration

This Python code calculates the thermonuclear reaction rate ($N_A\langle\sigma v\rangle$) over a temperature range of $T_9 = 0.001$–11 GK using the trapezoidal numerical integration method. It uses experimental cross-section data (in mb) and converts them into reaction rates through the Maxwell–Boltzmann weighted integral form of the reaction rate equation.

```
import numpy as np
from scipy.integrate import trapezoid

# Constants
NA = 6.02214076e23           # Avogadro's number (mol^-1)
PREFAC = 3.7313e7            # Prefactor (cm³ mol¹ s¹)
Z1 = 2                       # Alpha particle charge
m_alpha = 4.00260325415      # Alpha particle mass in amu

def reduced_mass_amu(m1_amu, m2_amu):
    """Compute reduced mass in atomic mass units (amu)."""
    return (m1_amu * m2_amu) / (m1_amu + m2_amu)

def reaction_rate_single_T9(T9, energies, cross_sections, mu_amu):
    """Compute <v> for a given T9 (GK)."""
    maxwellian = np.exp(-11.605 * energies / T9)
    integrand = energies * cross_sections * maxwellian
    integral = trapezoid(integrand, energies)
    rate = PREFAC * mu_amu**(-0.5) * T9**(-1.5) * integral
    return rate

def compute_reaction_rate_over_T9(energies, cross_sections, mu_amu):
    """Compute <v> across a temperature grid from 0.001 GK to 11 GK."""
    T9_values = np.linspace(0.001, 11, 50)
    rates = np.array([
        reaction_rate_single_T9(T9, energies, cross_sections, mu_amu)
        for T9 in T9_values
    ])
    return T9_values, rates

def main():
```

```
print("Enter energies in MeV separated by spaces:")
energies = np.array(list(map(float, input().strip().split())))

print("Enter cross sections in milli-barn separated by spaces:")
cross_sections = np.array(list(map(float, input().strip().split())))

if len(energies) != len(cross_sections):
    print("Error: The number of energies and cross sections must be equal.")
    return

print("Enter target atomic number (Z2):")
Z2 = int(input())

print("Enter target mass in atomic mass units (amu):")
mass_target = float(input())

mu_amu = reduced_mass_amu(m_alpha, mass_target)
print(f"\nReduced mass  = {mu_amu:.5f} amu\n")

T9_vals, rate_vals = compute_reaction_rate_over_T9(energies, cross_sections,
mu_amu)

# ---- Part 1: ln(<v>) ----
print(f"{'T9 (GK)':>10} {'ln(<v>) (cm³ mol¹ s¹)':>40}")
print("-" * 52)
for T9, rate in zip(T9_vals, rate_vals):
    if rate > 0:
        ln_rate = np.log(rate)
        print(f"{T9:10.3f} {ln_rate:40.6f}")
    else:
        print(f"{T9:10.3f} {'-inf':>40}")  # ln(0) undefined

print("\n" + "="*52 + "\n")

# ---- Part 2: Actual <v> ----
print(f"{'T9 (GK)':>10} {'<v> (cm³ mol¹ s¹)':>40}")
print("-" * 52)
for T9, rate in zip(T9_vals, rate_vals):
```

```
        print(f"{T9:10.3f} {rate:40.6e}")

if __name__ == "__main__":
    main()
```

This code outputs both the logarithmic and actual reaction rates, facilitating comparison between direct integration results and values obtained through logarithmic fits.

Appendix A6: Calculation of Cross-Section and $E\sigma(E)$ Using Polynomial $ln(S)$ Fit

This Python script calculates the astrophysical S-factor and corresponding cross-sections for the reaction ^{45}Sc(α,n)^{48}V using a polynomial fit to ln(S). The S-factor is obtained by exponentiating the fitted polynomial, which smooths the energy dependence of the cross-section and enables estimation of $E\sigma(E)$ values over a continuous energy grid.

```
import numpy as np

# === Physical constants ===
amu_to_MeV = 931.49410242   # 1 amu = MeV/c^2
e_squared = 1.44            # MeV·fm
hbar_c = 197.327            # MeV·fm

# === Projectile & Target ===
Z1 = 2                      # alpha particle charge
Z2 = 21                     # target charge (Scandium)
mass_projectile = 4.00260325415  # alpha in amu
mass_target = 44.95590828        # target in amu

# Convert to MeV
M_proj = mass_projectile * amu_to_MeV
M_target = mass_target * amu_to_MeV

# Reduced mass (in MeV/c^2)
mu = (M_proj * M_target) / (M_proj + M_target)

def lab_to_cm(E_lab, M_proj, M_target):
    return E_lab * M_target / (M_proj + M_target)

def compute_eta(Z1, Z2, mu, E_cm):
    return (Z1 * Z2 * e_squared / hbar_c) * np.sqrt(mu / (2 * E_cm))

# === Energy grid ===
Emax = 9.922
n = 96
E_lab = np.linspace(1e-6, Emax, n)
E_cm = lab_to_cm(E_lab, M_proj, M_target)
```

```
# === Polynomial fit for ln(S) ===
lnS = 0.0062*E_cm**3 - 0.2075*E_cm**2 + 1.0183*E_cm + 30.7
S = np.exp(lnS)  # S(E) in MeV·barn

# === Gamow factor ===
etas = compute_eta(Z1, Z2, mu, E_cm)
two_pi_eta = 2 * np.pi * etas
exp_minus_2pi_eta = np.exp(-two_pi_eta)

# === Compute (E) and E(E) ===
sigma = (S / E_cm) * exp_minus_2pi_eta   # (E) in barns
E_sigma_E = E_cm * sigma                 # in MeV·barn

# === Print results ===
print(f"{'E_lab (MeV)':>12} {'E_cm (MeV)':>12} {'':>12} {'2':>12} "
      f"{'lnS':>12} {'S-factor (MeV·barn)':>20} {'E(E) (MeV·barn)':>20}")
for e_lab, e_cm, eta, twopi, lnS_val, S_val, val in zip(
        E_lab, E_cm, etas, two_pi_eta, lnS, S, E_sigma_E):
    print(f"{e_lab:12.3f} {e_cm:12.3f} {eta:12.5f} {twopi:12.5f} "
          f"{lnS_val:12.5f} {S_val:20.5e} {val:20.5e}")
```

The cubic polynomial fit for $\ln(S)$ reproduces the smooth variation of S-factor with energy and provides a basis for generating theoretical $E\sigma(E)$ values. At lower energies, deviations between this polynomial-based result and directly computed S-factors become more prominent, indicating the need for refined low-energy fitting.

Appendix A7: Reaction Rate Calculation from $E\sigma(E)$ Data Using Trapezoidal Integration

This script calculates the thermonuclear reaction rate $\langle \sigma v \rangle$ as a function of temperature (T_9 in GK) using numerical integration of the product $E\sigma(E)$, obtained from astrophysical S-factor calculations or experimental data. The integration is performed using the trapezoidal rule.

```
import numpy as np

# --- Constants ---
mu = 3.67537  # reduced mass in amu
prefactor = 3.7313e7 / np.sqrt(mu)

# --- User input: Energies and E(E) values ---
E = np.array(list(map(float, input("Enter E values in MeV: ").split())))
E_sigma_E = np.array(list(map(float, input("Enter E(E) values in MeV·mbarn: ")
.split())))

# --- Reaction rate function ---
def reaction_rate(T9, E, E_sigma_E):
    integrand = (E_sigma_E*1000) * np.exp(-11.605 * E / T9)
    integral = np.trapz(integrand, E)
    return prefactor * T9**(-1.5) * integral

# --- Temperatures (GK) ---
T9_vals = [0.001, 0.225, 0.45, 0.674, 0.899, 1.123, 1.348, 1.572, 1.797, 2.021,
           2.246, 2.47, 2.695, 2.919, 3.144, 3.368, 3.593, 3.817, 4.041, 4.266,
           4.49, 4.715, 4.939, 5.164, 5.388, 5.613, 5.837, 6.062, 6.286, 6.511,
           6.735, 6.96, 7.184, 7.408, 7.633, 7.857, 8.082, 8.306, 8.531, 8.755,
           8.98, 9.204, 9.429, 9.653, 9.878, 10.102, 10.327, 10.551, 10.776, 11.0]

# --- Compute & print ---
print(f"{'T9 (GK)':>8} {'Rate (cm³ mol¹ s¹)':>25} {'log10(rate)':>15}")
print("-"*55)
for T9 in T9_vals:
    rate = reaction_rate(T9, E, E_sigma_E)
    log_rate = np.log(rate) if rate > 0 else -np.inf
    print(f"{T9:8.3f} {rate:25.6e} {log_rate:15.4f}")
```

The code integrates $E\sigma(E)$ weighted by the Maxwell–Boltzmann factor $\exp(-11.605E/T_9)$ to obtain $\langle\sigma v\rangle$ for a range of temperatures. The logarithmic values of the reaction rate are also computed for direct comparison with published data or polynomial fits. This enables cross-validation of numerically integrated and anti-logarithmic methods in assessing stellar reaction probabilities.

Appendix A8: Reaction Rate Calculation Using Gauss–Legendre Quadrature

This script computes the thermonuclear reaction rate $\langle \sigma v \rangle$ for given $E\sigma(E)$ data using **Gauss–Legendre numerical integration**, providing a more accurate alternative to the trapezoidal rule, especially for smoothly varying functions. The integration is performed over an interpolated energy grid between the experimental limits.

```
import numpy as np
from scipy.interpolate import interp1d

# --- Constants ---
mu = 3.67537  # reduced mass in amu
prefactor = 3.7313e7 / np.sqrt(mu)

# --- User input: Energies (MeV) and E(E) (MeV·barn) ---
E = np.array(list(map(float, input("Enter E values in MeV: ").split())))
E_sigma_E = np.array(list(map(float, input("Enter E(E) values in MeV·barn: ")
.split())))

# Convert E(E) from barns → millibarns (multiply by 1000)
E_sigma_E_mb = E_sigma_E * 1000.0

# Linear interpolation (avoids oscillations)
interp_func = interp1d(E, E_sigma_E_mb, kind="linear", fill_value="extrapolate")

# --- Gauss{Legendre Reaction Rate function ---
def reaction_rate_gauss(T9, E_min, E_max, interp_func, n_points=64):
    x, w = np.polynomial.legendre.leggauss(n_points)
    E_nodes = 0.5*(E_max-E_min)*x + 0.5*(E_max+E_min)
    w_nodes = 0.5*(E_max-E_min)*w

    E_sigma_vals = interp_func(E_nodes)
    E_sigma_vals = np.where(E_sigma_vals < 0, 0.0, E_sigma_vals)

    integrand = E_sigma_vals * np.exp(-11.605 * E_nodes / T9)
    integral = np.sum(w_nodes * integrand)

    return prefactor * T9**(-1.5) * integral
```

```
# --- Temperatures (GK) ---
T9_vals = [0.001, 0.225, 0.45, 0.674, 0.899, 1.123, 1.348, 1.572, 1.797, 2.021,
           2.246, 2.47, 2.695, 2.919, 3.144, 3.368, 3.593, 3.817, 4.041, 4.266,
           4.49, 4.715, 4.939, 5.164, 5.388, 5.613, 5.837, 6.062, 6.286, 6.511,
           6.735, 6.96, 7.184, 7.408, 7.633, 7.857, 8.082, 8.306, 8.531, 8.755,
           8.98, 9.204, 9.429, 9.653, 9.878, 10.102, 10.327, 10.551, 10.776, 11.0]

# --- Compute & print ---
print(f"{'T9 (GK)':>8} {'Rate (cm³ mol¹ s¹)':>25} {'log10(rate)':>15}")
print("-"*55)
for T9 in T9_vals:
    rate = reaction_rate_gauss(T9, np.min(E), np.max(E), interp_func, n_points=64)
    rate = max(rate, 0.0)
    log_rate = np.log10(rate) if rate > 0 else -np.inf
    print(f"{T9:8.3f} {rate:25.6e} {log_rate:15.4f}")
```

The script applies 64-point Gauss–Legendre quadrature to evaluate the rate integral:

$$N_A\langle\sigma v\rangle = 3.7313 \times 10^7 \mu^{-1/2} T_9^{-3/2} \int_0^\infty E\sigma(E) e^{-11.605E/T_9}\, dE$$

By interpolating $E\sigma(E)$ and using weighted quadrature nodes, this approach achieves higher numerical precision than the trapezoidal rule for smooth reaction functions.

Appendix A10: Cross–Section Plot with Visible Error Bars

This script reads experimental cross–section data along with their uncertainties and generates a plot of $\sigma(E)$ versus E including error bars. A connecting trend line is optionally added for visual guidance.

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
# User Input Section
# ------------------------------
E = np.array(list(map(float, input("Enter Energy values (MeV): ").split())))
sigma = np.array(list(map(float, input("Enter Cross-section values (mb): ")
.split())))
delta_sigma = np.array(list(map(float, input("Enter Cross-section errors ( in mb): ")
.split())))

# ------------------------------
# Plot Configuration
# ------------------------------
plt.figure(figsize=(8, 5))

plt.errorbar(
    E, sigma, yerr=delta_sigma,
    fmt='o', markersize=5, markeredgewidth=1.2, markeredgecolor='black',
    color='navy', ecolor='crimson', elinewidth=1.4, capsize=5, capthick=1.2,
    label='Experimental Data ( ± )'
)

plt.plot(E, sigma, linewidth=1.5, alpha=0.7, label='Trend Line')

plt.title("Cross-Section vs Energy with Error Bars", fontsize=14, fontweight='bold')
plt.xlabel("Energy (MeV)", fontsize=13)
plt.ylabel("Cross-Section (mb)", fontsize=13)
plt.grid(True, linestyle='--', linewidth=0.6, alpha=0.7)
plt.legend(fontsize=11, loc='best', frameon=True)
plt.tight_layout()
```

```
plt.savefig("cross_section_error_visible.png", dpi=300, bbox_inches='tight')
plt.show()
```

The use of error bars reflects the reported experimental uncertainty and ensures a more faithful interpretation and comparison with model predictions.

Appendix A11: S–Factor Plot with Error Bars

This Python script generates a plot of the astrophysical S–factor $S(E)$ as a function of energy. Experimental or computed values are visualized with error bars to reflect associated uncertainties.

```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# User Input
# ----------------------------
E = np.array(list(map(float, input("Enter Energy values (MeV): ").split())))
S_factor = np.array(list(map(float, input("Enter S-factor values (MeV·barn): ")
.split())))
delta_S = np.array(list(map(float, input("Enter S-factor errors (S in MeV·barn): "
.split())))

# ----------------------------
# Plotting
# ----------------------------
plt.figure(figsize=(8, 5))

# Line connecting points
plt.plot(
    E, S_factor,
    color='darkred', linewidth=2,
    marker='o', markersize=5,
    markerfacecolor='white', markeredgecolor='darkred',
    label='S-factor (calculated)'
)

# Error bars
plt.errorbar(
    E, S_factor, yerr=delta_S,
    fmt='none', ecolor='black', elinewidth=1.1,
    capsize=3, capthick=1
)
```

```
# Labels and title
plt.title("Astrophysical S-Factor vs Energy", fontsize=14, fontweight='bold')
plt.xlabel("Energy (MeV)", fontsize=12)
plt.ylabel("S-Factor (MeV·barn)", fontsize=12)
plt.grid(True, linestyle='--', linewidth=0.7, alpha=0.7)
plt.legend(fontsize=11, loc='best', frameon=True)
plt.tight_layout()

plt.savefig("S_factor_vs_energy_with_error_line.png", dpi=400, bbox_inches='tight')
plt.show()
```

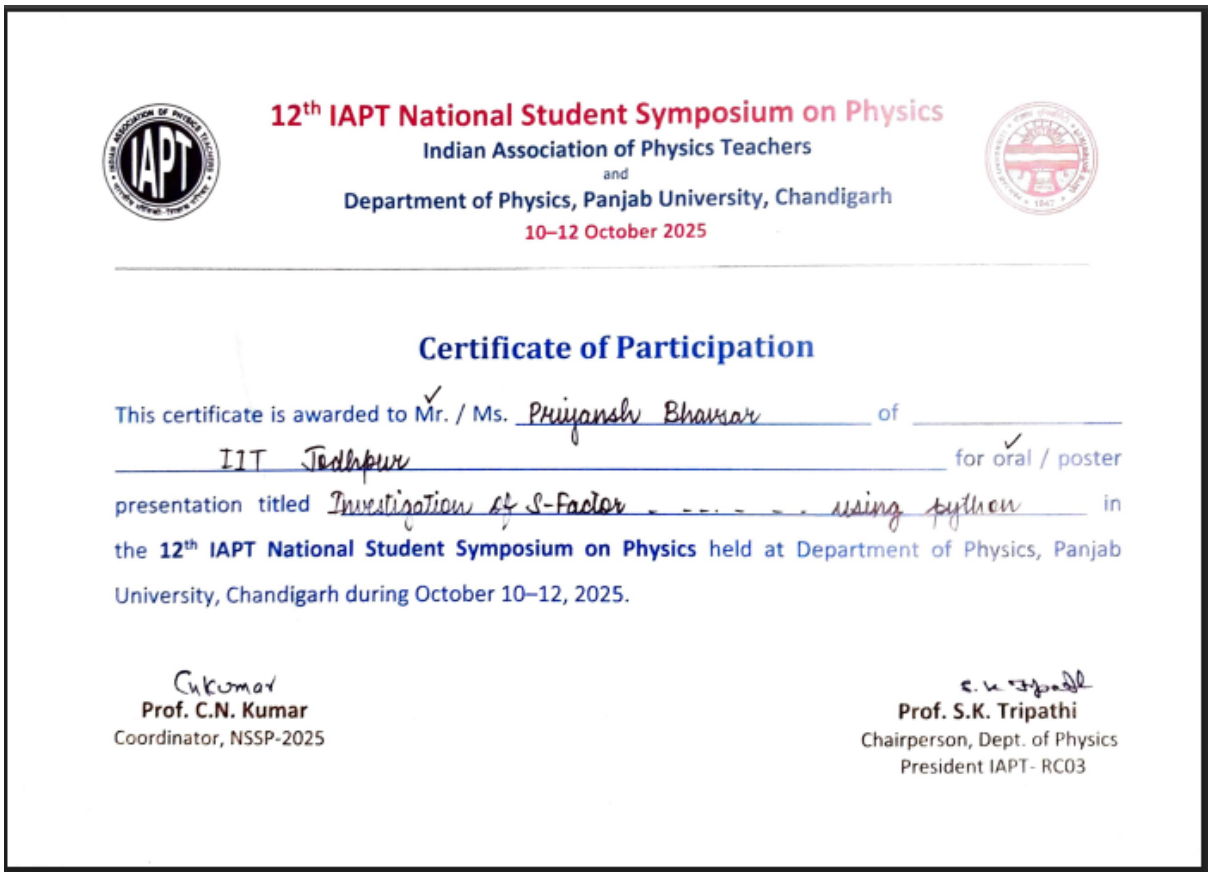
The inclusion of ΔS improves interpretation by visually conveying data precision and enabling better comparison with theoretical fits.

Appendix A12: Plot of Reaction Rate vs Temperature

This Python script compares the reaction rates obtained from direct numerical integration with those computed using anti–logarithmic polynomial expressions as a function of the stellar temperature T_9 (in GK). The y–axis is shown on a logarithmic scale to accommodate the large variation in rates.

```
import numpy as np
import matplotlib.pyplot as plt

# -------------------------------------
# User Input
# -------------------------------------
# Temperature values (T9 in GK)
T9 = np.array(list(map(float, input("Enter T9 (GK) values: ").split())))

# Reaction rates from integration (cm³ mol¹ s¹)
rate_computed = np.array(list(map(float, input("Enter Computed Reaction Rates: ")
.split())))

# Reaction rates from anti-log (cm³ mol¹ s¹)
rate_antilog = np.array(list(map(float, input("Enter Reaction Rates from Anti-log:
").split())))

# -------------------------------------
# Plotting
# -------------------------------------
plt.figure(figsize=(8, 5))

# Computed Reaction Rate
plt.plot(T9, rate_computed, color='#1f77b4', linewidth=2.2, marker='o',
         markersize=5, markerfacecolor='white', markeredgecolor='black',
         label='Computed (Integration)')

# Anti-log Reaction Rate
plt.plot(T9, rate_antilog, color='#d62728', linewidth=2.2, linestyle='--',
         marker='^',markersize=6, markerfacecolor='none', markeredgecolor='black',
         label='From Anti-log (Reference)')
```

```
plt.title("Reaction Rate vs Temperature (T)", fontsize=15, fontweight='bold')
plt.xlabel("Temperature T (GK)", fontsize=12)
plt.ylabel("Reaction Rate (cm³ mol¹ s¹)", fontsize=12)
plt.yscale('log')
plt.grid(True, linestyle='--', linewidth=0.7, alpha=0.7)
plt.legend(fontsize=11, loc='best', frameon=True)
plt.tight_layout()
plt.savefig("ReactionRate_vs_T9_Clean.png", dpi=400, bbox_inches='tight')
plt.show()

# NOTE:
# Values entered in this script will vary for each reaction system studied.
```



12th IAPT National Student Symposium on Physics

Indian Association of Physics Teachers

and

Department of Physics, Panjab University, Chandigarh

10–12 October 2025

Certificate of Participation

This certificate is awarded to Mr. / Ms. Priyansh Bhawar of IIT Jodhpur for oral / poster presentation titled Investigation of S-Factor using python in the **12th IAPT National Student Symposium on Physics** held at Department of Physics, Panjab University, Chandigarh during October 10–12, 2025.

Ckumar
Prof. C.N. Kumar
Coordinator, NSSP-2025

Prof. S.K. Tripathi
Chairperson, Dept. of Physics
President IAPT- RC03

Certificate of Participation in the National Student Symposium on Physics (NSSP) 2025.